

ENGINEERING LEADERSHIP KPIs — DIRECTOR & HEAD OF ENGINEERING

V1

What to measure, what it is paired with, and what never to measure at all
delivery · reliability · cost · people + e-commerce · streaming · social · chat · video calling · news feed · ★ marks the one that matters most per domain

1 · WHAT YOU ARE ACTUALLY ACCOUNTABLE FOR — AND THE TRAP AT THIS LEVEL

The four things a Director owns

Delivery	Does work reach production predictably, in small safe increments?
Reliability	Do the journeys that make money keep working?
Cost	Does unit economics improve as you scale?
People	Can the organisation still do this in two years?

The trap, and it catches most people at this level. Report only DORA metrics and your CEO hears nothing they care about — deployment frequency is not a business outcome. Report only revenue and conversion and your engineers cannot act on it. **Your job is the translation layer**: connecting a latency percentile to a conversion rate, and an error budget to a decision about when to ship.

So the structure that works is layered. A business outcome sits on top of a journey-level service objective, which sits on an engineering KPI, which sits on a team-level signal. Each layer explains the one above and is actionable by the layer below — panel 4 sets this out.

The one-line test for any proposed KPI

"If this number moves, what decision changes?" If nobody can answer, it is a dashboard ornament. Delete it.

Choosing them

- Pick five to eight. A director tracking forty KPIs is tracking none. Choose the ones tied to this year's constraint
- Retire them when the constraint moves. KPIs are instruments, not monuments. A metric nobody has acted on in two quarters is noise
- Every KPI gets a counter-metric — see panel 14. A metric optimised alone always produces a pathology
- Percentiles, not averages. The average request is fine, the p99 is the one losing you customers
- Trend over target in the first year. Direction and consistency beat an invented threshold
- Instrument the journey, not the service. Every service at 99.9% can still mean a broken checkout

Leading versus lagging, and why you need both. Lagging metrics — incident count, attrition, revenue impact — tell you what already happened and are what the business asks about. Leading metrics — error budget burn, PR review latency, on-call load, dependency age — tell you what is about to happen, and are the ones you can still act on. A dashboard of only lagging indicators is a rear-view mirror.

How this sheet is organised

Panels 2-7 Universal. These apply whatever you build — delivery, reliability, cost, security, tech health and people

Panels 8-13 Domain-specific. Six product types, each with the small number of KPIs that genuinely matter and one marked ★ as the headline

Panels 14-16 Counter-metrics, what to report upward, the anti-patterns, and an operating cadence

A note on benchmarks. This sheet deliberately gives no target numbers — no "deploy N times a day", no "p95 under N milliseconds". Those depend entirely on your domain, scale, regulatory context and architecture, and a borrowed number is worse than none because it manufactures false confidence. Measure your own baseline first, then set a target against it.

The honest training for your first ninety days: most organisations cannot measure half of this. Start by finding out which ones you genuinely cannot produce — that list is half the finding, and it is usually a more useful report than any of the numbers.

2 · DELIVERY — THROUGHPUT AND SAFETY, WHICH MUST BE READ TOGETHER

UNIVERSAL the four research-backed measures, plus the flow metrics that explain them

KPI	What it actually tells you
Deployment frequency	A proxy for batch size and confidence. Failing frequency almost always means fear, not less work — investigate it as a signal about the team, not the pipeline
Lead time for change	Report p50 and p90 together. The p50 flatters you; the p90 is where the painful path lives, and it is the one engineers experience
Change failure rate	Share of deploys causing degradation or requiring remediation. The safety half of the pair — deployment frequency without it is meaningless
Failed-deployment recovery time	How quickly you undo a bad change. Often a better investment than preventing the change from failing

Read the first and third together, always. High deployment frequency with a high change failure rate is reckless. Low frequency with a low failure rate is usually not safety — it is large batches that simply have not failed yet. The healthy pattern is both improving at once, which is the finding that made these four measures interesting in the first place.

Flow metrics — where the lead time actually goes

PR review latency	Usually the single largest hidden component of lead time, and the cheapest to fix. Measure it before buying any tooling
PR size distribution	Large PRs cause slow reviews, which cause larger PRs. Breaking that loop moves several other metrics at once
Work in progress per team	High WIP is context-switching made visible. Reducing it raises throughput without adding people
Flaky test rate	Erodes trust in the pipeline. Once engineers re-run failures by reflex, your safety net is gone
Build and test duration p50	Directly sets how small a batch can reasonably be
% work unplanned	Interrupt load. Above roughly a third, planning is fiction and your estimates will keep being wrong

The diagnostic sequence that works. Lead time is high — break it into coding, review wait, build, and release wait — in most organisations review wait dominates — fix that before anything else. It is a working-agreement change, not a budget request.

3 · RELIABILITY

journeys, not servers

KPI	Notes
SLO attainment per critical journey	The headline. One number per journey that the business recognises
Error budget burn rate	The most useful mechanism at your level. It converts reliability from an argument into a rule: budget spent means stabilise, budget healthy means ship faster
Incidents by severity, trend	Raw count is noise. The severity mix is the signal
MTTD — time to detect	Usually the worst of the three and the cheapest to improve. Most "slow recovery" is slow detection
MTTA — time to acknowledge	Tests your paging path, not your engineers
MTTR — time to restore	Restore, not root-cause. Those are different activities
% incidents self-detected	Versus reported by a customer. A blunt, honest measure of observability quality
Repeat-incident rate	Same cause recurring means postmortem actions are not landing
Postmortem action closure	Closure rate, not write-up rate. Unclosed actions are a promise nobody kept

The two-metric version, if you can only have two: SLO attainment on the revenue-critical journey, and percentage of incidents you detected before a customer told you. Between them they cover whether the product works and whether you would know if it did not.

Track toil separately — the share of time spent on manual, repetitive, automatable work. Sustained above roughly thirty percent and your engineering team has become a support desk, which shows up as attrition long before it shows up in delivery metrics.

4 · THE MEASUREMENT MODEL — FOUR LAYERS, AND WHAT EACH AUDIENCE SEEES

THE FOUR LAYERS — each explains the one above it and is actionable by the one below

1 · BUSINESS OUTCOME — what the CEO and the board care about Revenue, conversion, retention, cost as a share of revenue, regulatory exposure. You do not own these alone — but you must be able to show how engineering moved them.	each layer explains the one above
2 · JOURNEY SLO — the translation layer, and the most valuable thing you will build Checkout completes · message delivers · call connects · feed loads. Stated as an objective with an error budget, so reliability becomes a decision rule rather than an opinion.	
3 · ENGINEERING KPI — what you manage week to week Lead time, change failure rate, payment success rate, rebuffing ratio, unit cost, on-call load. These are yours. This is the layer this sheet is mostly about.	
4 · TEAM SIGNAL — diagnostic only, never a performance measure PR review latency, flaky test rate, build duration, WIP dependency age. Used by the team to improve itself. The moment these are used to rank people they stop being true.	

READING THE MODEL — six principles that decide whether a metrics programme works or quietly rots

- Every KPI needs a counter-metric. A number optimised alone always produces a pathology — deployment frequency without change failure rate produces recklessness, moderation speed without false-positive rate produces over-blocking. Panel 14 lists the pairs.
- Team-level signals must never become performance measures. The instant PR review latency appears in an appraisal, engineers optimise the metric rather than the work, and you lose both the metric and the trust that produced it. This is the fastest way to destroy a measurement programme.
- Percentiles, not averages — and the tail is the product. The mean request is always fine. The p99 is the customer who left. For anything user-facing, report p50 and p99 and treat the average as decoration.
- Measure the journey, not the service. Service-level availability multiplies: twenty services at 99.9% each do not give you a 99.9% checkout. Instrument the path the user actually takes, end to end, and let service metrics be diagnostic.
- Baseline before target. Borrowed benchmarks manufacture false confidence — another company's numbers encode their domain, scale and architecture, not yours. Measure for a quarter, then set a target against your own trend.
- A KPI nobody has acted on in two quarters should be deleted. Dashboards accumulate metrics the way codebases accumulate dead code, and an unread dashboard is worse than none because it creates the impression of oversight.

Useful interrogation of any metrics programme: which number would change a decision this week, who is the audience for each dashboard, what is the counter-metric for your headline KPI, and which of these can you not currently produce at all?

That last question is usually the most productive one in a new role — the metrics you cannot produce describe your observability and data gaps more precisely than any audit will.

On cadence: weekly for flow and incidents with the teams, monthly for domain KPIs with product, quarterly for the five that go upward. Reviewing everything at every cadence is how metrics become theatre — and the meeting that reviews forty numbers and changes nothing is the clearest sign the programme has stopped working.

WHICH NUMBERS TRAVEL UPWARD

TO THE BOARD OR CEO — five, quarterly Unit cost trend · SLO attainment on the revenue-critical journey · Peak readiness before a known event Flagged attrition · Security SLA adherence. Nothing else. Adding a sixth dilutes all five.	each layer explains the one above
TO YOUR PEERS IN PRODUCT — monthly Error budget status, which says what you can safely commit to · Lead time p90 · The domain KPI that maps to their goal · Unplanned work share, which explains why the roadmap moved	
TO YOUR OWN TEAMS — weekly, and visible to them without asking Everything. Teams see their own flow metrics, their own incidents, their own cost. Transparency downward, summary upward — the revenue is how metrics become a surveillance exercise and stop reflecting reality.	

5 · COST AND UNIT ECONOMICS — THE LANGUAGE THAT TRAVELS

HIGH LEVERAGE the fastest way to be taken seriously outside engineering

KPI	Why it earns attention
Unit cost per order, viewing hour, message, participant-metric	The single most useful number you can produce. It converts infrastructure into a figure finance already reasons about, and it makes an architecture improvement legible as a margin improvement
Infra cost as % of revenue, trend	Should decline as you scale. If it is flat, you have an architecture problem, not a procurement problem — and this is the chart that proves it either way
Cost per team and per service	Attribution must exist before optimisation is possible. Showing teams their own number reduces spend before any technical change
Cost of a percentage point of reliability	Makes the "why not five nines" conversation quantitative rather than cultural
Idle and over-provisioned capacity	Usually the largest single line, and the least contentious to reclaim

The framing that works with engineers: cost is a design constraint, not a procurement problem. A partitioning choice, a cache hit rate, a retry policy and an instance type are all cost decisions made by engineers — and they move the number far more than renegotiating the contract does.

Tech health — the debt that is not a metaphor

% services on supported versions	Runtimes, frameworks, databases, base images. Your deferred-maintenance liability in one number, and the one that eventually blocks a security fix
Dependency age — median and worst	A leading indicator. The worst case is usually the thing that will make an upgrade urgent and expensive later
Migration completion %	For each in-flight migration, with a date. Half-finished migrations are the most expensive state — you pay for both systems and get the benefits of neither
Services without an owner	Should be zero. Anything above zero is an incident waiting for a weekend
Toil %	Manual repetitive work. Predicts attrition earlier than any engagement survey

The number to watch for a stalled migration. Track completion percentage with a date, and escalate when it stops moving. A migration that is eighty percent done for three quarters is a strategic failure that will not surface in any other metric — delivery and reliability can both look healthy while it quietly doubles your operating surface.

6 · SECURITY & COMPLIANCE

measurable, not aspirational

KPI	Notes
Time to patch critical vulnerabilities	Against a stated SLA, and report adherence rather than the raw count. This is the one that goes upward
Open critical and high findings, ageing	The ageing distribution matters more than the total — a small number of very old findings is worse than many new ones
% services with scanning and SBOM	Coverage of the programme itself. Undetected services are where the surprise lies
Secret exposure incidents	Should be zero. Also measure time to detect and rotate — exposure happens, undetected exposure is the failure
% of production images signed and verified	Supply-chain posture, expressed as a percentage rather than a policy document
Access review completion	Standing privileged access is the finding auditors reach for first
Audit or certification findings, and repeat findings	A repeat finding is far more damaging than a new one — it says the previous remediation was cosmetic
Recovery objectives proven	Not stated — proven by a tested restore, with the measured time. An untested backup is a hypothesis

Report adherence, not activity. "Ninety-four percent of critical vulnerabilities patched within SLA, worst outstanding at eleven days" is a sentence an executive can act on. "We closed six hundred findings" is not — it says nothing about what remains.

The one that catches organisations out is proven recovery. Everyone has backups, far fewer have restored one this year and timed it. Make the measured restore time a KPI, because it is the number your recovery objective actually depends on — and the one that turns out to be wrong during an incident.

7 · PEOPLE & ORGANISATION

the leading indicators

KPI	Notes
Regretted attrition	Separate it from total attrition, which is nearly meaningless — losing a poor fit and losing your best architect are not the same event. Report regretted only
On-call load distribution	Pages per engineer per week, and out-of-hours share. Concentration here predicts your next resignation more reliably than any survey
Pages per on-call shift, trend	Rising means alerting quality is degrading. Failing to near zero may mean alerts were silenced rather than fixed
Time to first meaningful contribution	Measures your onboarding and documentation, not the hire
Internal mobility rate	People moving between teams is a health signal. Zero mobility means teams are hoarding
Bus factor on critical systems	Count systems with a single knowledgeable owner. This is a risk register, not a vanity metric
Engineering time on planned work	The inverse of interrupt load, and the honest explanation for a missed roadmap
Hiring pipeline health	Time to hire, offer acceptance rate, funnel drop-off by stage

On-call load is the most under-used metric on this sheet. It is a leading indicator of attrition, an honest measure of system health, and a direct consequence of engineering decisions — unlike an engagement score, which tells you sentiment has already deteriorated. It is also one of the few numbers where the fix and the measure sit in the same team's hands.

Never measure individuals with any metric on this sheet. Commits, lines, story points, review counts — all of them reward volume over judgement and destroy the collaboration that produces good systems. Team and system level only.

8 · E-COMMERCE

conversion is the frame

KPI	Why
★ Checkout success rate	The headline. Every fraction of a percent is direct revenue, and it is the number the business already understands
Payment success rate by gateway and method	Split it — isolates a failing provider from a failing checkout. Aggregate success rate hides a single broken method
Duplicate order rate	Idempotency failures. A correctness bug that costs refunds and trust, and one nobody notices until a customer complains
Overall rate	Inventory consistency in one number — and a direct customer-experience failure
Product and search page latency p95	Tie it explicitly to conversion. The latency-to-revenue link is the most reliable argument you have for performance work
Search zero-result rate	Usually the largest untapped conversion lever, and cheap to measure
Cart abandonment from technical causes	Separated from ordinary abandonment. Only the technical share is yours
Order-to-fulfilment success	Where asynchronous failures hide — the order succeeded and then quietly did not progress
Peak headroom	Capacity divided by forecast peak, before a sale event. The metric that prevents the annual incident
Refund and chargeback SLA	Operational load and a regulatory exposure

The conversation this enables: "Checkout success is 97.2%. Recovering one point is worth X in revenue, and the top cause is one payment provider's timeout behaviour." That is an engineering finding stated as a business case — and it will be funded.

9 · STREAMING

quality of experience

KPI	Why
★ Rebuffering ratio	The dominant quality metric. Time spent buffering as a share of viewing time. It correlates with abandonment more strongly than any other technical
Time to first frame startup time, p50 and p95	Drives abandonment before playback even begins. The p95 is where the lost viewers are
Video start failure rate	Distinct from rebuffering — the stream never began at all. A different failure with a different cause
Exit before video start	Your startup experience expressed as lost viewers rather than milliseconds
Average delivered bitrate	Quality actually served, versus quality encoded. The gap is your delivery problem
CDN offload ratio	Share served from edge rather than origin. Drives both cost and latency
Cost per GB delivered and per viewing hour	Your largest bill, and the two ways to talk about it
DRM licence failure rate	Small number, disproportionate support load, and highly visible to customers
Encoding throughput and cost per content-hour	Pipeline efficiency, and the constraint on catalogue growth
Peak concurrent streams vs capacity	Especially for live events, where the peak is known in advance and unforgiving

Segment quality metrics by device, geography and network. An acceptable aggregate rebuffering ratio routinely conceals one platform or one region having an unusable experience — and that segment is where your churn is concentrated.

10 · SOCIAL PLATFORM

content in, content out

KPI	Why
★ Post publish success rate	Never lose a user's content. A failed publish is the single most damaging experience on a social product
Media upload success rate and processing time	The highest-friction path in most social apps, and disproportionately mobile-network dependent
Notification delivery rate and latency	Split iOS and Android — platform behaviour differs enough that an aggregate number misleads
Graph query latency p99	Follower and friend traversals are the hardest thing you scale. The p99 is the account with many connections
Moderation time to action, by severity	Increasingly a regulatory metric, not merely an operational one. Track the severe categories separately
Moderation false-positive rate	The counter-metric. Without it you optimise speed into over-blocking, and creators leave
Abuse detection precision and recall	Report both. Either alone is trivially gamed in opposite directions
Appeal volume and overturn rate	A high overturn rate means your enforcement is wrong, not that users are complaining
Availability weighted by traffic	An outage at peak is not equivalent to one at the quietest hour; weight accordingly

Moderation is where engineering KPIs meet regulatory exposure. Time to action on severe categories is increasingly something you must be able to evidence to a regulator. Build the measurement before you are asked for it — retrofitting it under a deadline is materially harder.

11 · CHAT & MESSAGING

correctness before latency

KPI	Why
★ Message delivery success rate	Must approach one hundred percent. Measure it explicitly end to end — do not infer it from service availability
Message loss rate	Should be zero. Discovering a non-zero number you did not know about is the worst finding available in this domain
Delivery latency p50 / p95 / p99	The p99 is the conversation that felt broken
Ordering violation rate	Out-of-order messages are a correctness bug users notice immediately and report as the product being broken
Duplicate delivery rate	The other half of exactly-once, and equally visible
Connection drop rate and reconnect success	Mobile networks make this your real reliability story — drops are inevitable, failed reconnects are not
Offline sync time and success	The experience after a tunnel or a flight, and a common source of perceived loss
Push notification delivery rate	Per platform. Often the weakest link in perceived delivery
Media attachment success rate	Separate from text; different failure profile entirely
Group fan-out latency by group size	Large groups are where the architecture is tested

This is the one domain where correctness ranks above latency. Users forgive a message arriving a second late; they do not forgive one arriving twice, out of order, or not at all. Order your KPIs accordingly — and make loss rate a number someone looks at weekly, not one you assume is zero.

12 · VIDEO CALLING

connect first, then quality

KPI	Why
★ Call join success rate	Failing to connect is far worse than degraded quality. A poor call is an annoyance; a call that will not start is a product failure
Call setup time	Time from join to media flowing. Directly shapes the perception of the product
Call drop rate	Separated from user-ended calls — which requires deliberate instrumentation
Mean opinion score	The standard audio-quality measure and your headline experience number
Packet loss, jitter, round-trip time	Distributions, never averages. The average conceals exactly the sessions that were bad
Video freeze rate and duration	What users actually describe as a bad call
Audio-video sync drift	Small drift is more irritating to users than lower resolution
Relay usage share	Your main cost driver. Relay calls cost far more than direct peer connections — track the share and what pushes it up
Cost per participant-minute	The unit economics of the product
Join success by network type	Corporate networks and restrictive firewalls are where joins fail

Rank these deliberately: join success, then drop rate, then quality. Teams routinely optimise resolution and bitrate while a measurable share of calls never connect at all — and the second group churns, while the first merely complains.

13 · NEWS FEED AND TIMELINE — THE FAN-OUT PROBLEM, MEASURED

Twitter-shaped products · STAFF the tail is the architecture, so measure it separately

KPI	Why
★ Timeline generation latency p90	The defining technical constraint of the product. Everything in the architecture exists to hold this number down
Feed freshness post — visible to followers	What your fan-out strategy actually optimises, and the number users perceive as the product being live
Fan-out completion for high-follower accounts	Track the tail separately from the median. The celebrity case is a different system from the ordinary one, and averaging them hides both
Timeline cache hit rate	Drives latency and cost simultaneously — the highest-leverage single number after p99 latency
Ranking inference latency	Usually the dominant contributor to the p99 once caching is working
Write amplification ratio	Fan-out writes per post. Capacity and cost expressed as one number
Hot key and hot partition incidents	Your recurring structural failure. Count them; the trend tells you whether mitigation is working
Search and trend indexing lag	How stale discovery is relative to posting

Why the tail deserves its own metric here more than in any other domain. A feed product has two populations that behave nothing alike — the ordinary account with a modest following, and the small number of accounts whose every post triggers enormous fan-out. A single median latency figure describes neither, and the second population is where both your incidents and your most visible users are. Report them as separate series.

Engagement metrics you should be careful with

Time in feed, session length
Engagement rate
Driven by ranking and content, not by reliability. Useful as context for capacity planning, misleading as an engineering KPI

What you can legitimately own: that the feed loads quickly, is fresh, is available, ranks within its latency budget, and costs a predictable amount per user per day. Engagement is the product organisation's outcome, and being precise about that boundary is part of the job — it protects you when engagement dips for reasons that have nothing to do with engineering.

Capacity planning here is event-driven. Feed products have sharp, sometimes predictable peaks — a major news event, a sporting final, a product launch. Peak headroom against a forecast belongs on your dashboard alongside the latency numbers, for the same reason it does in e-commerce.

14 · COUNTER-METRICS

every KPI needs a guard

Any metric optimised alone produces a pathology. This is not cynicism about people — it is what happens when you point an organisation at a single number. The fix is structural: publish each KPI with the metric that would degrade if it were gamed, and review the pair together.

Primary KPI	Must be paired with
Deployment frequency	Change failure rate — otherwise you are rewarding recklessness
Lead time	Escaped defect rate — speed achieved by skipping review is not speed
Incident count trending down	MTTD — you may simply be detecting fewer, not causing fewer
MTTR trending down	Repeat-incident rate — fast restoration of the same fault is not improvement
Latency	Cost per request — latency is always purchasable with money
Cost reduction	SLO attainment — savings taken out of reliability arrive as an outage later
Moderation time to action	False-positive rate — speed alone produces over-blocking
Test coverage	Escaped defect rate — coverage is trivially inflated with assertion-free tests
Throughput or velocity	Rework and defect rate — output is not outcome
Utilisation	Lead time — high utilisation lengthens queues; a fully loaded system is a slow one
Alert volume down	% incidents self-detected — alerts may have been silenced rather than fixed

Make the pairing visible on the dashboard itself, side by side rather than on separate pages. The discipline is not in choosing good metrics — it is in never showing one without its guard, so the trade-off is present in every conversation rather than discovered in a retrospective.

15 · WHAT NEVER TO MEASURE — AND THE FAILURE MODES OF A METRICS PROGRAMME

Never measure these	Why it causes harm
Metric creep	Rewards verbosity and punishes deletion — the most valuable change a senior engineer makes
Lines of code	Trivially inflated by spelling work. Measures activity, never outcome
Commits or PR counts	Initiates within one sprint of being watched. It is a planning aid for a team, and worthless the moment it leaves the team
Story points or velocity as performance	
Any individual developer metric	Destroys collaboration faster than anything else here. Helping a colleague, reviewing carefully and mentoring all reduce your personal numbers
Test coverage as a target	Goodhart's law in its purest form — you get tests that assert nothing
Raw uptime without journey SLOs	99.9% while checkout is broken is a number that misleads everyone who reads it
Hours worked or presence	Measures compliance, not contribution, and selects for the wrong people

How a good programme rots

Failure	What it looks like
Metric creep	Forty numbers on a dashboard, none reviewed. Add a retirement rule when you add the metric
Team signals used for appraisal	The fastest way to destroy the programme. Engineers optimise the measure, and you lose both the number and the trust that produced it
Vanity reporting upward	Only improving metrics reach the board. The first genuine problem then arrives as a surprise, and your credibility goes with it
Averages everywhere	Every percentile-shaped problem becomes invisible
Borrowed benchmarks	Another organisation's numbers encode their domain and architecture. Chasing them produces motion without direction
Measuring what is easy	The available number displaces the important one. Build-duration dashboards while nobody measures checkout success
No counter-metrics	Every headline number drifts toward its pathology within two quarters

Signals your programme is working

- A metric changed a decision this quarter — the only test that matters
- Teams look at their own numbers without being asked, and argue about them
- You have retired a KPI, not only added ones
- A number went the wrong way and you reported it upward anyway — and the conversation was productive
- Product plans against your error budget rather than negotiating reliability after the fact
- An engineer cites a KPI in a design review unprompted

The single question to ask of any dashboard: "when did this last change what we did?" If the honest answer is never, the dashboard is providing the appearance of oversight rather than oversight — which is worse than having none, because it stops anyone looking further.

And the one cultural rule that protects everything above: metrics are used to improve systems, never to rank people. State it explicitly, repeatedly, and act consistently with it — the first time a number appears in a performance conversation, every number you collect afterwards becomes less true.

16 · OPERATING CADENCE & FIRST-90-DAYS CHECKLIST

turning a list into a practice

- Named the business outcome engineering is accountable to this year
- Critical user journeys identified and agreed with product
- An SLO defined per critical journey, with an error budget
 - Error budget policy written — what happens when it is spent
- Five to eight KPIs chosen, tied to this year's constraint
- Every KPI has a named counter-metric shown beside it
- Every KPI has an owner and a review cadence
- Baseline measured before any target was set
- Percentiles used for everything user-facing
 - The metrics you cannot produce are listed — that list is a finding
- Delivery, deployment frequency, lead time p90, change failure rate, recovery time
- PR review latency measured before buying delivery tooling
- Unplanned work share tracked
- Reliability: SLO attainment, budget burn, MTTD separated from MTTR

- % incidents self-detected rather than customer-reported
- Postmortem action closure tracked, not just write-ups
- Repeat-incident rate visible
- Unit cost defined for your domain and trended
- Cost attributed per team, and teams can see it
- Infra cost as % of revenue charted over time
- Stalled migrations surfaced with a completion % and a date
- Services without an owner: zero
- Security SLA adherence reported, not finding counts
- Recovery objectives proven by a timed restore
- Regretted attrition separated from total
- On-call load distribution reviewed monthly
- Bus factor recorded for critical systems
- Domain KPIs chosen from the relevant panel, with one headline

- Domain KPIs segmented by device, region or network where it matters
- Peak headroom tracked ahead of known events
- Weekly with teams — flow, incidents, on-call
- Monthly with product — error budget, lead time, domain KPI
- Quarterly upward — the five that travel, good news and bad
- Teams see everything: upward reporting is summarised
- No individual-level metric exists anywhere in the system
- Team signals explicitly excluded from performance conversations
- A retirement rule exists for metrics nobody acts on
 - At least one KPI retired in the last two quarters
- A bad number was reported upward without being softened
- You can name a decision a metric changed this quarter

The one that summarises the sheet: name the business outcome you are accountable to, the journey SLO that supports it, the engineering KPI that moves that SLO, and the counter-metric that stops the KPI being gamed. If you can state that chain for your top priority, the rest of this sheet is detail. If you cannot, no number on it will help — because measurement without that chain is reporting, not management.